

TIN DANG HUYNH TRUNG

Technical Architect

@ tindang.ht97@gmail.com

+84 937 684 229

Ho Chi Minh City, Viet Nam

TinDang97

TinDang97

I'm a low-key developer: a cup of coffee, a hard problem, and something useful at the end of it.

Technical architect, ten years in software — mostly Python, plus Rust and TypeScript. I've had machine learning in production since 2019: camera analytics and a vector search engine I wrote myself, then fintech pricing and language models, and for the last two years the infrastructure under LLM products — gateways, routing and failover, retrieval, agent memory, and metering that reconciles against what providers actually bill. I've been CTO of an AI startup, led teams of up to fourteen, and still keep my hands on the hard parts. I'd rather measure than claim, so retrieval quality is gated in CI and inference is tuned against a P95. An agent writes most of my code now; deciding whether it's correct is still my job, and I published the method I use for that.

PROFESSIONAL EXPERIENCE

Technical Architect

Trustify Technology

Jan 2024 – Present

Ho Chi Minh City, Vietnam

I architect and lead client platforms across Vietnam and Myanmar — transport, fintech, real estate and investment data for clients including Yoma Bank and VinaCapital — and wrote the company's engineering career ladder.

- **Real-time ML serving at the edge** • *intelligent transport — architect & sole engineer, Sep 2025 – Mar 2026*
40 camera streams are more video than a city network can carry, so I moved inference to the roadside and ship only events. **85 ms P95 inference**: YOLO on TensorRT in half precision behind Triton with dynamic batching, falling back to ONNX Runtime where no GPU is present. **75% less backhaul**, since H.265 transcoding and frame filtering also run at the edge; events cross mutually authenticated Kafka into PyFlink, load-tested to **71K events a second**.

Python · FastAPI · YOLO · TensorRT · Triton · ONNX Runtime · Kafka · PyFlink · K3s · ArgoCD · DuckDB

- **Estate-finance platform & PCI boundary** • *real-estate group — architect & sole engineer, Jul – Sep 2026*
I replaced a live PHP/Laravel system whose balances nobody could audit with **one append-only double-entry ledger**: thirty modules in eight releases over four weeks, with **235 legacy business rules graded** against it before any customer moved. **PCI scope is one service** — cardholder data has its own service, database and network, enforced by the build.

Python 3.12 · FastAPI · async SQLAlchemy 2 · PostgreSQL (schema per tenant) · Redis · 2C2P · AWS ECS · Envoy

- **Also delivered** • *data, migration and payments*
A Databricks lakehouse for VinaCapital that cut reporting **from two days to two hours**; the rollback path of an IoT device platform's per-customer migration, where a rehearsal showed the document store, cache and object archive did not revert with Postgres; 2C2P card payments for a mobile-wallet product; and a microfinance lending MVP for Yoma Bank in Java 21 and Spring Boot.

CTO – Co-founder

ClickAI Viet Nam

Jun 2024 – Oct 2025 (concurrent)

Ho Chi Minh City, Vietnam

I co-founded ClickAI (clickai.vn) and built its conversational AI platform from scratch — multi-modal chat, image and audio generation, community — to 40K+ users across Southeast Asia inside four months, leading twelve to fourteen people and owning routing, billing and deployment myself.

- **Model platform & retrieval** • *20+ providers*
20% lower API spend: an orchestration layer routes each request by policy — task complexity, compliance, cost against quality — tested in A/B runs. After a provider went hard-down and users felt it before routing shifted, I **probed every provider continuously** and marked one down before traffic could reach it. Behind the same gateway sat **our own open-weight LLMs on vLLM**, LangChain orchestration, and **80% faster retrieval** from hybrid search over Weaviate and Qdrant with citation tracking; a Graphiti knowledge graph on FalkorDB carries context between conversations.

Python · FastAPI · LangChain · vLLM · Weaviate · Qdrant · FalkorDB · Graphiti · ComfyUI · Replicate · fal.ai · ElevenLabs

- **Billing, metering & production** • *sole author — 99.9% uptime, 1K+ concurrent sessions*
I built the wallet, plan and referral engine over a month-partitioned ledger that meters every LLM call. **A reconciliation check I wrote caught a 30% billing drift** no alert had seen — a retry loop was re-sending work — and I keyed idempotency on the source event so a retry cannot double-count. I authored the 13-service Kubernetes deployment, held **sub-2s responses**, moved PostgreSQL onto a three-node Patroni cluster when it saturated, and **halved the hosting bill**; when a release regressed, my own instrumentation caught it before any customer did.

Python · FastAPI · PostgreSQL (partitioned, Patroni) · Redis · Kubernetes · Helm · Azure · Prometheus · Next.js 14

Founder & Principal Engineer

Pilot Space (open-source AI infrastructure)

📅 Jan 2026 – Present

📍 Remote

My own open-source line, each project published with its benchmark: **Lunaris** (agent memory; recall over 100K documents in 19–22 ms), **laya-codex** (code retrieval with a re-ranker I fine-tuned; right files in an agent's first answer up from 73% to 88%), **Moon** (Redis-compatible store with vector, full-text and graph search; 1.62× Redis on GET), **Hydroa** (multi-tenant LLM gateway) and **ADD** (my method for trusting agent-written code). Details and links are on the last page.

EARLIER EXPERIENCE (2018 – 2024)

- **Senior Data Engineer, Asia Commercial Bank (ACB)** • Aug 2023 – Jun 2024
80% less manual verification on a data lake carrying 5M+ transactions a day, from a data-quality framework I built on Spark across six dimensions; my FastAPI services fed 100+ internal applications. Cloudera · PySpark · Dagster · Iceberg · FastAPI
- **Software Data Engineer, Vietmoney Corp** • Mar 2021 – Aug 2023
Pricing, language and vision models for an appraisal and lending platform carrying 10M+ daily transactions at 99.9% uptime. **Appraisal fell from 15 minutes to 5 seconds** on price forecasting at 96% accuracy; a Vietnamese entity model fine-tuned from BERT reached **96% F1**; a 20K-node asset graph links products to listings; and a distributed crawler I designed **doubled throughput** to 1,000+ pages an hour across 50+ marketplaces. I also started the company's open-source face-anti-spoofing. Python · PyTorch · BERT · scikit-learn · Prophet · FAISS · Scrapy · Pub/Sub · BigQuery · Airflow · GCP
- **Software Developer (AI), Dai Phat Solutions** • Aug 2019 – Mar 2021
Camera AI – vector search engine & distributed inference; architect of the camera-AI product. Recognising faces across many cameras needs a similarity lookup on every frame, and the first pipeline showed video three to five seconds late – too slow for a security alert.
 - **Delay cut from 3–5 s to 1 s** across 16 cameras at 15 FPS each: I tuned FFmpeg decoding, moved JPEG coding to TurboJPEG, compressed arrays on the wire with Blosc and reused pre-allocated memory for vectors.
 - **Wrote the vector search engine** on FAISS: flat and IVF indexes with training, half-precision GPU indexes movable between GPU and CPU, persistence and de-duplication on insert; served over Kafka request-and-reply, with vectors in Redis.
 - **Optimised model inference.** I exported the detection and face-recognition models from PyTorch to TorchScript, ONNX Runtime and TensorRT, ran them in half precision, and batched frames from several cameras onto one GPU.
 - **Designed it as a distributed system.** Decoding, detection, embedding and search workers pass frames over Kafka, Pulsar and ZeroMQ, so capacity grows by adding workers and one failed worker does not stop a camera.
 - **92% recognition accuracy over 1,000+ identities** removed 90% of manual processing; incidents fell 40% across 20+ clients.Python · PyTorch · RetinaFace · InsightFace · FAISS · TensorRT · ONNX Runtime · Kafka · Pulsar · ZeroMQ · Redis · FFmpeg · Docker
- **Full-stack Developer, LogiGear Corporation** • Jun 2018 – Mar 2019
Built an Ethereum wallet and smart-contract application in Node.js and React.

TECHNICAL SKILLS

LLM systems: multi-provider gateways with policy routing, continuous health probing and failover across OpenAI, Anthropic, Gemini, Bedrock, Azure and OpenRouter; open-weight models served on vLLM; token metering and cost reconciliation; LangChain, MCP clients and servers, agent memory; evaluation harnesses and regression gates in CI

Retrieval & vector search: embeddings, hybrid keyword-and-vector search with rank fusion, cross-encoder re-ranking and fine-tuning, knowledge graphs (Graphiti); FAISS (IVF, GPU indexes), HNSW, Weaviate, Qdrant, FalkorDB

ML engineering & serving: PyTorch, YOLO, RetinaFace and InsightFace, BERT fine-tuning, scikit-learn; TensorRT, Triton, ONNX Runtime, TorchScript, OpenVINO and CoreML with dynamic batching and half precision; FFmpeg video pipelines; edge deployment on K3s

Languages & backend: Python (typed, async), Rust, TypeScript / Node.js, Java, SQL; FastAPI, SQLAlchemy 2, Pydantic, pytest, Tokio, Next.js / React; hexagonal architecture, CQRS, event sourcing, multi-tenancy, outbox, idempotency, PCI boundaries, OAuth / OIDC

Data, storage & cloud: PostgreSQL (Patroni, partitioning, schema per tenant), Redis, MongoDB, DynamoDB, DuckDB, Delta Lake; Kafka, PyFlink, PySpark, Airflow, BigQuery, Databricks; Kubernetes, Helm, Docker, Envoy, OpenTelemetry, Prometheus; AWS, Azure, GCP

EDUCATION, TEACHING & CERTIFICATIONS

Education: BSc Computer Science (High-Quality Program), Ton Duc Thang University, 2015–2019; NewAI Lab research intern

Teaching & speaking: Design Patterns instructor, Ton Duc Thang University (2022); data-analysis mentor, MindX (2023); speaker, AI Workshop, FPT University (2025)

Certifications: Google Cloud Technical Series – AI Agents (2025); AWS Machine Learning Foundations (2022)

Languages: English (advanced) – the working language of my code, documentation and client work; Vietnamese (native)

Everything on this page is public unless marked otherwise, and each figure comes from the benchmark published in its repository. Pilot Space (github.com/pilot-space) is my own open-source line of AI infrastructure, built since January 2026.

Moon • github.com/pilot-space/moon • [docs pilot-space.github.io/moon](https://docs.pilot-space.github.io/moon) • *moondb* on crates.io and PyPI

A Redis-compatible in-memory data store written from scratch in Rust, so existing Redis clients connect unchanged: RESP2 and RESP3, 250+ commands. Vector search (HNSW), BM25 full-text search with hybrid rank fusion, a property-graph engine and transactions that hold across all of them ship in one binary. Against Redis 7.0 it measures 1.62× the throughput on GET and 1.32× on SET with 8–22% less memory per key, and the command families where it loses are published beside the wins. Thirty releases to v0.8.9; single-node production-grade, clustering in alpha.

Lunaris • github.com/pilot-space/lunaris • [docs pilot-space.github.io/lunaris](https://docs.pilot-space.github.io/lunaris) • *lunaris-memory* on crates.io, *@pilot-space/lunaris* on npm

An agent-memory engine built on Moon: ingestion, extraction, embeddings, hybrid search and re-ranking behind one API, with the embedding and re-ranking models running in-process so recall makes no network call. Recall across 100,000 documents per scope returns in 19–22 ms at the median and under 25 ms at P99 on a single shard; beyond 100,000 documents it is published as unvalidated. A CI gate scores evidence recall on LongMemEval-S on every retrieval change and fails the build when recall drops. I wrote that gate after finding its predecessor had never once run — all 200 recorded executions were startup failures.

laya-codex • github.com/pilot-space/laya-codex • [model huggingface.co/tindang/laya-code](https://model.huggingface.co/tindang/laya-code)

Ranked code retrieval for coding agents. The repository is split into chunks with tree-sitter (14 languages), keyword search finds candidates, and a cross-encoder re-ranker I fine-tuned on git history — where a commit's changed code is the answer to its message — orders them and places the code in the agent's first turn through hooks and MCP. On 60 paired tasks across 3 repositories the right files appeared in the first answer 88% of the time, up from 73%; tool calls per session fell from 5.9 to 2.3, session time by 19% and cost by 14.8%. On 60 further tasks the cost gain was not significant, and the README says so.

Hydroa • github.com/pilot-space/hydroa

A multi-tenant LLM gateway: one OpenAI-compatible endpoint over OpenAI, Anthropic, Gemini, Bedrock, Azure and OpenRouter. Every row is scoped to a tenant, each tenant's provider keys are encrypted at rest, providers sit behind circuit breakers with health-aware fallback, and a cost ledger reconciles what was metered against what the provider billed.

ADD • github.com/pilot-space/ADD • [book pilot-space.github.io/ADD](https://book.pilot-space.github.io/ADD) • *@pilot-space/add* on npm, *pilot-space-add* on PyPI

The spec-first method behind the projects above: the contract is frozen before any code exists, the failing test is committed before the fix, and a build is accepted on a recorded test run rather than a diff that reads well. On 300 real bugs from open-source projects it resolved 226 against 215 for the same agent unassisted, and saw the tests pass before shipping in 274 cases against 53 — at 1.8× the cost. The README states that the gain in bugs fixed is not yet proven.

yowo • github.com/TinDang97/yowo • on PyPI

Production YOLO11 and YOLO26 inference and export. It detects the hardware and picks the backend in order — TensorRT, ONNX Runtime on CUDA, CoreML, OpenVINO, CPU — reconnects dropped RTSP streams and tracks objects with ByteTrack. Extracted from the transport platform on page one.

rjson • github.com/TinDang97/rjson • *pyrjson* on PyPI (experimental)

JSON for Python written in Rust against the CPython C API: parsing 1.29× and serialising 1.46× the speed of orjson over ten reference cases, with byte-identical output.

dpsutil & the search-engine client • github.com/TinDang97/DPS_Util • github.com/TinDang97/DPS_SearchEngine_Client

The library and client behind the camera-AI product at Dai Phat (2019–2021): Kafka and Redis wrappers, Blosc array compression, TurboJPEG coding, a pre-allocated vector pool, and the client for the FAISS search service — flat and IVF indexes, half-precision GPU indexes, GPU-to-CPU moves. 44 releases on PyPI.

Also • smaller and earlier work

Helios, a Rust agent harness with 12 provider backends, a native MCP client and server and sandboxed WebAssembly extensions (private, unreleased) • **Pilot Space**, an AI-augmented software-delivery platform (github.com/pilot-space, MIT) • **fastapi-multi-tenant**, a multi-tenant FastAPI boilerplate (github.com/TinDang97/fastapi-multi-tenant) • **Velos**, a GPU traffic microsimulation in Rust (github.com/pilot-space/velos) • **face-anti-spoofing**, started at Vietmoney (github.com/vietmoney/face-anti-spoofing) • **face_recognition**, RetinaFace and InsightFace (github.com/TinDang97/face_recognition).